

PROYECTO 01 DE 10

MercaAndes Omnicanal

Visibilidad de inventario y ventas en tiempo casi real para una cadena retail omnicanal

Ficha del proyecto	
Organización (ficticia)	MercaAndes S.A.S. — cadena de 140 tiendas físicas y un marketplace en línea
Sector	Retail / Comercio electrónico
Duración	6 semanas
Equipo sugerido	3 a 4 integrantes
Costos AWS	No hay presupuesto predefinido: el equipo debe encontrar y justificar la solución de costo óptimo.
Alcance esperado	Solución completa end-to-end: desde la generación e ingesta de datos hasta el tablero, desplegada en AWS.
Tipo de documento	Enunciado de problema. No incluye solución: el equipo define arquitectura, infraestructura, recursos y costos.

Nota para el equipo. Este documento describe un problema de negocio, sus reglas, sus fuentes de datos y cómo se aceptará el trabajo. Deliberadamente **no** indica cómo resolverlo. Cualquier decisión técnica (qué servicio, cuántos recursos, qué patrón, qué topología) es responsabilidad del equipo y debe justificarse por escrito.

Contenido

1. Contexto de negocio
 2. Problema a resolver
 3. Objetivos del negocio
 4. Actores y usuarios
 5. Fuentes de datos
 6. Reglas de negocio
 7. Requerimientos funcionales
 8. Requerimientos no funcionales
 9. Temas del programa que el proyecto debe evidenciar
 10. Lo que el equipo debe definir
 11. Entregables
 12. Criterios de aceptación
 13. Preguntas de negocio para el tablero
 14. Restricciones y supuestos
 15. Fuera de alcance
 16. Hitos sugeridos
 17. Rúbrica de evaluación
-

1. Contexto de negocio

MercaAndes es una cadena de retail con presencia en cinco países de la región andina. Vende a través de 140 tiendas físicas, un marketplace propio y vendedores terceros (sellers) que publican en su plataforma. En los últimos dos años el canal digital pasó de representar el 12% al 38% de las ventas.

Cada canal nació con su propio sistema: los puntos de venta (POS) escriben en una base de datos relacional por país, el marketplace opera sobre un sistema heredado que exporta archivos planos cada noche y los sellers informan su inventario a través de una API. Nadie en la compañía tiene hoy una única vista de “cuánto vendimos y cuánto nos queda”.

La dirección comercial aprobó un proyecto para construir una plataforma de datos que unifique ventas, inventario y experiencia del cliente, y que habilite decisiones de reabastecimiento y promociones con información de no más de 15 minutos de antigüedad.

2. Problema a resolver

La empresa pierde ventas por quiebres de stock que no se detectan a tiempo y, al mismo tiempo, acumula sobre-inventario en otras tiendas. Las promociones se lanzan sin saber si hay inventario suficiente y los reportes de ventas se consolidan manualmente en hojas de cálculo con dos días de retraso.

Se requiere un sistema que reciba eventos de venta y de inventario de todos los canales, los normalice, aplique las reglas comerciales de la compañía, genere alertas operativas y alimente una capa analítica consultable por las áreas comercial, logística y de servicio al cliente.

2.1 Dolores actuales

- Quiebres de stock detectados por el cliente antes que por la empresa (estimado: 6% de ventas perdidas).
- Tres versiones distintas del “total vendido” según el área que lo calcule.
- Promociones activadas sobre productos sin inventario, generando cancelaciones y malas reseñas.
- Sellers que reportan inventario desactualizado y no son penalizados.
- Reportes comerciales que dependen de una sola persona y tardan 48 horas.

3. Objetivos del negocio

- Reducir en 40% los quiebres de stock no detectados en los primeros tres meses de operación.
- Tener una única fuente de verdad para ventas netas, devoluciones e inventario disponible.
- Disponer de alertas de quiebre inminente con anticipación mínima de 24 horas.
- Medir y clasificar el desempeño de los sellers con criterios objetivos.
- Entregar un tablero comercial diario sin intervención manual.

4. Actores y usuarios

Actor	Qué necesita del sistema
Gerente comercial	Ventas netas por canal, categoría, país y periodo; efectividad de promociones.
Analista de reabastecimiento	Alertas de quiebre inminente y sugerencias de traslado entre tiendas.
Seller (tercero)	Consultar su propio puntaje de desempeño y sus penalizaciones vía API.
Servicio al cliente	Estado de un pedido y su historial de eventos en una sola consulta.
Equipo de datos	Monitorear cargas, reprocesar periodos y auditar la calidad de los datos.

5. Fuentes de datos

El sistema debe integrar todas las fuentes siguientes. Las marcadas como **simuladas** deben ser construidas por el equipo siguiendo las especificaciones de la sección 5.2; su diseño interno es decisión del equipo.

ID	Fuente	Tipo	Acceso / formato	Frecuencia	Volumen
F1	Brazilian E-Commerce Public Dataset by Olist	CSV Kaggle	kaggle.com/datasets/olistbr/brazilian-ecommerce (9 archivos CSV)	Carga histórica única	~100 mil pedidos, ~1,5 M filas totales
F2	BD transaccional POS por país	BD simulada	PostgreSQL, un esquema por país. Poblada mediante seed.	Continua	≥ 2 M ventas en 12 meses
F3	API de inventario de sellers	API simulada	REST JSON paginado, autenticación por token	Cada 10 min	≥ 3.000 sellers, 50 mil SKUs
F4	Eventos de pedidos del marketplace	Stream simulado	Mensajes JSON (pedido creado, pagado, enviado, entregado, cancelado, devuelto)	Continuo	≥ 20 eventos/segundo o pico
F5	Tasas de cambio	API pública	Frankfurter API (api.frankfurter.app) — tasas BCE	Diaria	1 registro/moneda/día
F6	Calendario de festivos	API pública	Nager.Date (date.nager.at) por país	Anual	~20 registros/país/año
F7	Plan de promociones	Archivo plano	Excel/CSV cargado por el área comercial en almacenamiento de objetos	Semanal	< 5.000 filas

5.1 Detalle de fuentes externas

- F1 se usa como histórico del canal digital. El equipo debe mapear sus entidades (orders, order_items, payments, reviews, sellers, products, customers, geolocation) al modelo de MercaAndes y documentar el mapeo.
- F5 no cubre todas las monedas andinas; el equipo debe definir y documentar una estrategia para las monedas faltantes (COP, PEN, BOB, CLP según disponibilidad).
- F6 debe consultarse para CO, PE, EC, BO y CL; los festivos afectan las reglas de reabastecimiento.

5.2 Especificación de fuentes simuladas

Obligatorio — Seed de datos. Todos los datos de las bases de datos simuladas deben generarse mediante un **seed** (scripts de siembra) versionado en el repositorio. No se aceptan datos cargados a mano ni volcados (dumps) sin su seed.

El seed debe: (1) usar una semilla fija para que los datos sean reproducibles; (2) ser idempotente (ejecutarlo varias veces no duplica datos); (3) permitir parametrizar el volumen (modo desarrollo pequeño y modo completo); (4) respetar la integridad referencial y un catálogo coherente con las APIs y eventos simulados; (5) inyectar los defectos de calidad de la sección 5.3 con una tasa configurable; (6) ejecutarse tanto en local como contra la base de datos en AWS.

- F2: generador configurable (semilla fija) que produzca ventas con estacionalidad semanal y picos en festivos y días de promoción; debe poder correr en modo histórico (backfill) y en modo continuo.
- F3: servicio REST propio que exponga el inventario por seller y SKU, con paginación, límite de tasa (rate limit) y respuestas de error intermitentes (HTTP 429 y 503) en al menos 3% de las solicitudes.
- F4: productor de eventos que respete el ciclo de vida de un pedido, pero que también emita eventos fuera de orden y duplicados.
- Todas las fuentes simuladas deben compartir un catálogo coherente de productos, tiendas y sellers.

5.3 Problemas de calidad de datos conocidos

Los datos llegarán con los siguientes defectos. Las fuentes simuladas deben **inyectarlos deliberadamente** con una tasa configurable:

- SKUs escritos con distinto formato entre canales (mayúsculas, guiones, ceros a la izquierda).
- Eventos duplicados (mismo id de evento) y eventos que llegan hasta 6 horas tarde.
- Precios en cero o negativos en el 0,5% de las ventas POS.
- Sellers que reportan inventario negativo.
- Fechas en distintas zonas horarias sin indicación explícita.

6. Reglas de negocio

Las reglas son obligatorias. Cada regla debe estar trazada a código y a al menos una prueba automatizada.

ID	Regla de negocio
RN-01	La venta neta es el valor pagado menos descuentos, menos devoluciones aceptadas, excluyendo impuestos. Se reporta en USD usando la tasa del día de la venta.
RN-02	Una devolución solo es válida si ocurre dentro de 30 días calendario desde la entrega; fuera de ese plazo se registra como excepción y no afecta la venta neta.
RN-03	El inventario disponible = inventario físico – unidades reservadas por pedidos pagados no despachados – unidades en tránsito de salida.
RN-04	Un SKU está en “riesgo de quiebre” cuando su inventario disponible cubre menos de 3 días de venta promedio de los últimos 14 días. Si en los próximos 5 días hay festivo, el umbral sube a 5 días.
RN-05	No se puede activar una promoción sobre un SKU cuya cobertura de inventario sea inferior a 7 días; el intento debe quedar registrado y rechazado.
RN-06	Un pedido cancelado después de despachado se trata como devolución, no como cancelación.
RN-07	Puntaje de seller (0–100): 40% cumplimiento de tiempos de despacho, 30% calificación promedio de reseñas, 20% exactitud de inventario reportado, 10% tasa de cancelación inversa.
RN-08	Un seller con puntaje < 60 durante 2 semanas consecutivas pasa a estado “en observación”; con < 45 durante 2 semanas, a “suspendido”. Un seller suspendido no puede recibir nuevos pedidos.
RN-09	Exactitud de inventario del seller: porcentaje de pedidos que no se cancelaron por falta de stock en los últimos 30 días.
RN-10	Los eventos duplicados deben descartarse; los eventos tardíos (hasta 72 h) deben incorporarse y recalcular los agregados del día afectado.
RN-11	Los datos personales del cliente (nombre, documento, correo) no pueden llegar a la capa analítica sin seudonimizar.
RN-12	Una tienda que no reporta ventas durante 2 horas en horario comercial genera una alerta operativa.

7. Requerimientos funcionales

ID	Requerimiento	Prioridad
RF-01	Ingerir las siete fuentes descritas en la sección 5, con históricos y cargas incrementales.	Alta
RF-02	Exponer una API de consulta de estado de pedido con su línea de tiempo de eventos.	Alta
RF-03	Exponer una API para que cada seller consulte su puntaje, estado y penalizaciones (solo los propios).	Alta
RF-04	Exponer una API para registrar solicitudes de promoción y responder aprobada/rechazada según RN-05.	Alta
RF-05	Calcular diariamente el riesgo de quiebre por SKU y tienda y notificar al área de reabastecimiento.	Alta
RF-06	Recalcular automáticamente los agregados cuando llegan eventos tardíos.	Media

ID	Requerimiento	Prioridad
RF-07	Permitir reprocesar cualquier rango de fechas sin intervención en la base de datos.	Alta
RF-08	Generar el modelo analítico para ventas, inventario y desempeño de sellers.	Alta
RF-09	Registrar y exponer métricas de calidad de datos por fuente y por carga.	Media
RF-10	Detectar tiendas silenciosas (RN-12) y emitir alerta.	Media

8. Requerimientos no funcionales

ID	Atributo	Requerimiento medible
RNF-01	Latencia	p95 de las APIs de consulta ≤ 300 ms con 50 usuarios concurrentes.
RNF-02	Frescura	Los datos de ventas visibles en la capa analítica con ≤ 15 min de retraso para el día en curso.
RNF-03	Disponibilidad	APIs con 99,5% mensual; tolerar la caída de una zona de disponibilidad.
RNF-04	Durabilidad	Ningún evento de pedido confirmado puede perderse, incluso si un consumidor se reinicia.
RNF-05	Escalabilidad	Soportar 5x el volumen de eventos en temporada (Black Friday) sin cambios de código.
RNF-06	Seguridad	Tráfico cifrado; bases de datos sin acceso público; secretos fuera del código.
RNF-07	Eficiencia de costos	La solución debe ser la de menor costo que cumpla los demás requerimientos; justificar cada servicio.

9. Temas del programa que el proyecto debe evidenciar

El proyecto debe demostrar el uso de los temas del programa. Se indica **qué evidencia** se espera, no cómo implementarla. Temas obligatorios en este proyecto: **20 de 20**.

#	Tema	Evidencia esperada	Estado
1	FastAPI I — APIs REST	Exponer al menos una API REST documentada en OpenAPI/Swagger con operaciones CRUD, parámetros de ruta y consulta, y contratos de request/response explícitos. En este proyecto: APIs de pedido, seller y promociones; además la API simulada de inventario (F3).	Obligatorio
2	FastAPI II — ORM y buenas prácticas	Persistencia relacional mediante ORM, validación de datos con modelos tipados, manejo de errores con códigos HTTP correctos y una estructura de proyecto por capas que el equipo debe justificar.	Obligatorio
3	Docker I — Fundamentos	Cada componente propio empaquetado en una imagen reproducible, con variables de entorno, .dockerignore y evidencia de optimización (tamaño de imagen antes/después).	Obligatorio

#	Tema	Evidencia esperada	Estado
4	Docker II — Multi-container	Levantar el ecosistema completo en local con un solo comando, con redes, volúmenes persistentes y comunicación entre servicios.	Obligatorio
5	AWS I — Fundamentos	Cuenta/rol con IAM de mínimo privilegio, región y zonas elegidas y justificadas, uso de AWS CLI y un análisis de costos del proyecto.	Obligatorio
6	AWS II — Networking + EC2	Diseño de red propio (VPC, subredes, tablas de rutas, gateways, Security Groups) y al menos un recurso EC2 con justificación de su propósito.	Obligatorio
7	AWS III — S3 + RDS	Almacenamiento de objetos con políticas y clases de almacenamiento justificadas, y base de datos administrada con estrategia de backups y conexión segura.	Obligatorio
8	AWS IV — ECR + ECS	Imágenes publicadas en un registro privado y al menos un servicio desplegado sobre un clúster de contenedores con su definición de tarea versionada.	Obligatorio
9	AWS V — Fargate + ALB	Servicios en cómputo serverless de contenedores, expuestos detrás de un balanceador con health checks y logs centralizados.	Obligatorio
10	RabbitMQ	Comunicación asíncrona con productores, consumidores, exchanges, colas, routing keys, confirmaciones (ACK) y mensajes persistentes. En este proyecto: Eventos de ciclo de vida del pedido con enrutamiento por tipo de evento y manejo de duplicados.	Obligatorio
11	Airflow I — Fundamentos	Orquestación de los procesos batch mediante DAGs con dependencias, reintentos, programación y parámetros.	Obligatorio
12	Airflow II — Celery + RabbitMQ	Ejecución distribuida de tareas del orquestador con workers, broker y base de metadatos, levantada en contenedores.	Obligatorio
13	Spark I — Fundamentos	Jobs de procesamiento distribuido con esquemas explícitos (no inferidos) y uso consciente de transformaciones y acciones.	Obligatorio
14	Spark II — Data Processing	Transformaciones de negocio con selección, filtros, columnas derivadas, agregaciones, joins y funciones sobre los datos del proyecto. En este proyecto: Cálculo de venta neta, cobertura de inventario y puntaje de sellers.	Obligatorio
15	Spark III — ETL + Performance	Pipeline ETL con particionamiento, formato columnar, caché donde aplique y evidencia medible de optimización (tiempos, shuffle, planes de ejecución). En este proyecto: Reproceso por eventos tardíos sin reescribir todo el histórico.	Obligatorio
16	Spark + S3 + ECS/Fargate	Jobs de procesamiento contenedorizados que leen y escriben en almacenamiento de objetos y se ejecutan en la nube sobre contenedores.	Obligatorio
17	CI/CD I — Aplicaciones	Flujo Git con ramas y Pull Requests, pipeline que ejecute lint, pruebas, build de imágenes, publicación en registro y despliegue.	Obligatorio

#	Tema	Evidencia esperada	Estado
18	CI/CD II — Data Engineering	Pipeline de datos con pruebas de jobs, validación de DAGs, controles de calidad de datos y versionamiento de artefactos.	Obligatorio
19	EKS — Introducción	Despliegue de al menos un componente en Kubernetes (Pods, Deployments, Services, namespaces) y un análisis comparativo ECS vs EKS para este caso. En este proyecto: Desplegar la API de sellers en Kubernetes y comparar con su versión en ECS.	Obligatorio
20	Arquitectura + Power BI + Proyecto Final	Arquitectura end-to-end documentada, Data Lake por capas, modelo dimensional, consultas analíticas, tablero en Power BI, observabilidad y presentación ejecutiva.	Obligatorio

10. Lo que el equipo debe definir

No existe una arquitectura "correcta" esperada. Se evalúa la **coherencia** entre las reglas de negocio, los requerimientos no funcionales, el costo y las decisiones tomadas. La solución debe ser **completa y end-to-end**: generación de datos (seed y simuladores), ingesta, procesamiento, almacenamiento, exposición por API, analítica y operación, todo desplegado y funcionando en AWS.

10.1 Decisiones generales

- Diseño de la solución completa end-to-end: cómo se generan, ingieren, procesan, almacenan, exponen y analizan los datos, y cómo se opera todo en producción.
- Arquitectura de solución end-to-end (componentes, flujos de datos, patrones de integración síncronos y asíncronos) con diagrama propio.
- Infraestructura en AWS: servicios, región, zonas de disponibilidad, diseño de red, seguridad perimetral y de identidad.
- Dimensionamiento de recursos: CPU, memoria, almacenamiento, número de réplicas/workers/executors, y la evidencia que soporta cada número.
- Estimación de costos mensuales por ambiente (desarrollo y producción) usando AWS Pricing Calculator, desglosada por servicio, con supuestos explícitos. No hay un monto definido: el equipo debe proponer la opción de costo óptimo y demostrar que lo es.
- Estrategia de datos: capas del Data Lake, formatos, particionamiento, retención, modelo dimensional y granularidad de las tablas de hechos.
- Estrategia de despliegue, ambientes, manejo de secretos, versionamiento y rollback.
- Estrategia de observabilidad: qué métricas, logs y alarmas son necesarias y qué umbrales disparan una acción.
- Registro de decisiones de arquitectura (ADR) con alternativas evaluadas y la razón de descarte de cada una.

10.2 Preguntas específicas de este caso

El documento de arquitectura debe responder explícitamente:

- ¿Cómo garantiza la arquitectura que un evento duplicado o tardío no altere la venta neta ya reportada de forma incorrecta?
- ¿Qué componentes necesitan alta disponibilidad multi-AZ y cuáles pueden tolerar interrupciones? ¿Cuánto cuesta cada decisión?
- ¿Cómo se escala para Black Friday y cómo se vuelve al estado normal sin pagar capacidad ociosa?
- ¿Dónde y cómo se seudonimizan los datos personales para cumplir RN-11?
- ¿Cómo se aísla la información entre sellers en la API?

10.3 Plantilla mínima de costos mensuales

El equipo debe completar (y ampliar) esta tabla. Los valores se dejan en blanco a propósito.

Servicio AWS	Configuración / supuesto	Uso mensual estimado	Costo Dev (USD)	Costo Prod (USD)
Total mensual				

*No se define un presupuesto. El equipo debe buscar la solución **óptima en costo** para los requerimientos del caso, comparar alternativas y demostrar por qué su propuesta no sobredimensiona ni subdimensiona los recursos.*

11. Entregables

ID	Entregable	Descripción	Semana
E-01	Documento de arquitectura	Diagrama(s) de arquitectura lógica y física end-to-end, descripción de componentes, flujos y justificación. Incluye mínimo 5 ADR. Versión 1 en S1 y versión final en S6.	S1–S6
E-02	Repositorio de código	Repositorio Git con README, estructura por componentes, convenciones de ramas y evidencia de Pull Requests revisados.	S1–S6
E-03	Seed de datos	Scripts de seed versionados que poblan todas las bases de datos simuladas: semilla fija, volumen parametrizable, idempotentes y con inyección configurable de defectos de calidad.	S1
E-04	Ambiente local reproducible	Todo el ecosistema levanta en local con un único comando documentado, incluida la ejecución del seed.	S2
E-05	API(s) documentadas	Contrato OpenAPI exportado y colección de pruebas (Postman/Bruno/HTTPie) con casos felices y de error.	S2
E-06	Diseño de infraestructura AWS	Diagrama de red, inventario de recursos, políticas IAM y justificación de cada servicio.	S3

ID	Entregable	Descripción	Semana
E-07	Análisis de costos mensuales	Enlace/export de AWS Pricing Calculator + tabla de costos por servicio y ambiente, supuestos, alternativas comparadas y justificación de por qué la solución propuesta es la de costo óptimo.	S3 y S6
E-08	Pipelines de datos	DAGs y jobs de procesamiento distribuido versionados, parametrizados e idempotentes.	S4
E-09	Pipelines CI/CD	Pipelines de aplicación y de datos funcionando, con evidencia de ejecuciones exitosas y fallidas.	S5
E-10	Diccionario de datos y modelo dimensional	Descripción de cada tabla/campo, linaje de datos y diagrama del modelo dimensional.	S6
E-11	Tablero Power BI	Archivo .pbix y capturas, respondiendo las preguntas de negocio de la sección correspondiente.	S6
E-12	Runbook operativo	Cómo desplegar, monitorear, reprocesar datos, escalar y recuperarse de fallas.	S6
E-13	Informe de rendimiento	Mediciones antes/después de optimizaciones en procesamiento distribuido y en la API.	S5
E-14	Demo end-to-end y presentación final	Presentación ejecutiva (15 min) + demo en vivo end-to-end (15 min): un dato generado por el seed o por una fuente viaja por todo el sistema desplegado en AWS hasta el tablero + preguntas (10 min).	S6
E-15	Matriz de trazabilidad	Tabla RN/RF → componente → prueba automatizada que la valida.	S6
E-16	Simulación de temporada alta	Prueba de carga a 5x con resultados, cuellos de botella y costo del escenario.	S6

12. Criterios de aceptación

12.1 Criterios específicos del caso

ID	Criterio de aceptación (verificable)
CA-01	Dado un pedido con eventos duplicados y fuera de orden, la API de estado devuelve la línea de tiempo correcta y sin duplicados.
CA-02	Una devolución a los 31 días de la entrega no reduce la venta neta y aparece en el reporte de excepciones.
CA-03	Una solicitud de promoción sobre un SKU con 5 días de cobertura es rechazada con código HTTP y mensaje de negocio claros.
CA-04	Un seller no puede consultar el puntaje de otro seller (respuesta 403 o 404 según lo defina el equipo y lo justifique).
CA-05	Un evento que llega 48 horas tarde modifica los agregados del día correspondiente tras la siguiente ejecución programada.
CA-06	Al detener un consumidor de eventos durante 10 minutos y reiniciarlo, no se pierde ningún mensaje.

ID	Criterio de aceptación (verificable)
CA-07	La alerta de riesgo de quiebre se genera para un caso de prueba con festivo próximo usando el umbral de 5 días.
CA-08	La capa analítica no contiene ningún dato personal en claro (verificado con una consulta de control).
CA-09	La carga histórica de F1 queda en formato columnar particionado y el equipo demuestra la mejora de tiempo de consulta frente al CSV original.
CA-10	El estado de un seller cambia a “suspendido” en el caso de prueba definido y la API de pedidos rechaza asignarle nuevos pedidos.

12.2 Criterios transversales

ID	Criterio de aceptación (verificable)
CT-01	El ecosistema completo levanta en local con un único comando y un README permite a una persona externa ejecutarlo en menos de 30 minutos.
CT-02	Ninguna credencial, llave o contraseña está en el repositorio ni en las imágenes; el pipeline de CI falla si detecta secretos.
CT-03	Toda la infraestructura desplegada está etiquetada (proyecto, ambiente, responsable) y el equipo demuestra que puede destruirla y recrearla.
CT-04	El pipeline de CI bloquea el merge a la rama principal si fallan lint, pruebas o validación de DAGs.
CT-05	El análisis de costos mensuales compara al menos dos alternativas de arquitectura, cada línea tiene un supuesto verificable y el equipo justifica por qué su propuesta es la óptima para los requerimientos.
CT-06	Todas las bases de datos simuladas se pueblan exclusivamente mediante el seed: al ejecutarlo dos veces con la misma semilla se obtienen exactamente los mismos datos, sin duplicados.
CT-07	La solución funciona end-to-end en AWS: en la demo se genera un dato en una fuente y se muestra su recorrido por todas las capas hasta el tablero de Power BI.
CT-08	Los procesos batch son idempotentes: re-ejecutar el mismo periodo no duplica ni corrompe datos.
CT-09	Existe al menos una alarma configurada y demostrada en vivo ante una falla provocada.
CT-10	El tablero de Power BI responde todas las preguntas de negocio listadas y los números cuadran con una consulta de control presentada por el equipo.

13. Preguntas de negocio para el tablero

El tablero en Power BI (alimentado desde la capa analítica que el equipo diseñe) debe responder:

- ¿Cuál es la venta neta en USD por país, canal y categoría, diaria, semanal y mensual?
- ¿Qué SKUs y tiendas están en riesgo de quiebre hoy y cuál es la venta en riesgo?
- ¿Qué promociones fueron rechazadas por inventario y cuánto se habría vendido?
- ¿Cuál es la distribución de sellers por estado y cómo evoluciona su puntaje?

- ¿Cuál es la tasa de devoluciones por categoría y el porcentaje de devoluciones fuera de plazo?
- ¿Cómo impactan los festivos en la venta por país?
- ¿Cuál es la calidad de datos por fuente (duplicados, tardíos, rechazados)?

14. Restricciones y supuestos

- Toda la infraestructura debe desplegarse en una única cuenta AWS y una sola región, elegida y justificada por el equipo.
- Se deben usar exclusivamente datos públicos o simulados; ningún dato personal real.
- Los recursos deben poder apagarse fuera del horario de pruebas para controlar costos.
- El dataset de Kaggle es de Brasil (BRL); el equipo decide cómo integrarlo con el resto de países.

15. Fuera de alcance

- Modelos de pronóstico de demanda con machine learning.
- Integración con pasarelas de pago reales.
- Aplicación web o móvil para usuarios finales.

16. Hitos sugeridos

Semana	Bloque del programa	Resultado esperado al cierre
Semana 1	Descubrimiento y diseño	Análisis de reglas de negocio y fuentes; arquitectura end-to-end v1 y ADRs iniciales; modelo de datos operativo; seed de las bases de datos simuladas funcionando.
Semana 2	Backend y contenedores	Fuentes simuladas y API(s) propias funcionando con persistencia; ecosistema multi-contenedor completo en local con un solo comando.
Semana 3	Fundamentos AWS	Diseño de red y seguridad, almacenamiento, base de datos administrada, imágenes en registro y primeros servicios desplegados; análisis de costos v1.
Semana 4	Mensajería, orquestación y procesamiento	Flujo asíncrono de eventos, DAGs ejecutándose en modo distribuido y primeros jobs de procesamiento distribuido.
Semana 5	Rendimiento, CI/CD y Kubernetes	ETL optimizado ejecutándose en la nube, CI/CD de aplicación y de datos, componente desplegado en Kubernetes, informe de rendimiento.
Semana 6	Integración end-to-end y cierre	Integración completa en AWS, modelo dimensional, Power BI, observabilidad, análisis de costos final, runbook, demo end-to-end y presentación.

17. Rúbrica de evaluación

Dimensión	Qué se evalúa	Peso
Comprensión del negocio y reglas	Las reglas de negocio están implementadas y probadas; el equipo explica su impacto.	15%
Arquitectura y decisiones	Arquitectura coherente, ADRs con alternativas reales, trade-offs claros.	15%
Infraestructura AWS y costos	Red, seguridad e IAM correctos; costos estimados, realistas y optimizados.	15%
Backend y mensajería	APIs robustas, contratos claros, manejo de errores, mensajería confiable.	10%
Ingeniería de datos	Orquestación, procesamiento distribuido, calidad de datos, rendimiento.	20%
DevOps y CI/CD	Automatización real de pruebas, build y despliegue de aplicación y datos.	10%
Analítica y comunicación	Modelo dimensional, tablero útil, presentación clara y demo funcional.	15%

Penalizaciones: credenciales expuestas (−15%), infraestructura sin destruir tras la demo que genere costos (−10%), demo que no ejecuta en vivo (−10%), estimación de costos sin supuestos (−5%).